Питання для самостійного опрацювання

Тема 6. Параметричний поліморфізм. Контейнери

Виконав: Вівчар Вадим Вікторович
Група: АЛК-43

Нижче наведено відповіді на контрольні запитання за темою параметричного поліморфізму і контейнерів стандартної бібліотеки C++.

1. Які складові стандартної бібліотеки C++?

До стандартної бібліотеки C++ входять класи і функції загального призначення. У частині STL основними складовими є контейнери, ітератори, алгоритми, алокатори, адаптери та функціональні об'єкти. Крім цього, стандартна бібліотека містить підсистеми введення-виведення, роботи з рядками, файлами, винятками, багатопотоковістю, регулярними виразами та інші засоби.

2. Чи можливо в програмі на C++ використати бібліотеку C?

Так, у програмі на C++ можна використовувати бібліотеку C. Для цього підключаються відповідні заголовки, наприклад або . На практиці частіше рекомендують використовувати сучасні C++-аналоги, однак сумісність із бібліотекою C зберігається.

3. Чим відрізняється клас complex від pair?

Клас std::complex є спеціалізованою математичною абстракцією комплексного числа. Він має дійсну та уявну частини і підтримує арифметичні операції, характерні саме для комплексних чисел. Натомість std::pair - це універсальний шаблон для збереження двох значень, які розглядаються як пара first і second, без спеціальної предметної семантики.

4. Які типи символів підтримує клас string?

Клас std::string є спеціалізацією шаблону basic_string і працює з однобайтовими символами типу char. Також у стандартній бібліотеці існують інші спеціалізації: wstring для wchar_t, u16string для char16_t і u32string для char32_t.

5. Що таке конкатенація?

Конкатенація - це операція зчеплення рядків, тобто об'єднання двох або більше рядків в один. У C++ для класу string конкатенація часто виконується за допомогою операторів + та +=.

6. Які відмінності size від capacity в класі vector?

size() повертає фактичну кількість елементів, які зараз містяться у векторі. capacity() показує, скільки елементів вектор може зберігати без додаткового

перевиділення пам'яті. Тому значення capacity може бути більшим за size.

7. Як запрограмувати двовимірний масив на основі класу vector?

Двовимірний масив на основі vector реалізують як вектор векторів. Наприклад, конструкція vector> matrix; дозволяє створити таблицю, де кожен елемент зовнішнього вектора є окремим рядком.

8. Які недоліки у контейнерів типу vector?

Основним недоліком vector є неефективність вставки і видалення елементів у середині контейнера, оскільки для цього потрібно зсувати інші елементи. Крім того, при збільшенні розміру може відбуватися перевиділення пам'яті, що призводить до копіювання або переміщення даних та до інвалідності деяких ітераторів і посилань.

9. Чим контейнер list відрізняється від vector?

Контейнер vector зберігає елементи у суцільному блоці пам'яті і забезпечує швидкий доступ за індексом. Контейнер list є двозв'язним списком, тому прямий доступ за індексом у ньому відсутній, зате вставка і видалення елементів у відомій позиції виконуються зручніше. Таким чином, vector краще підходить для довільного доступу, а list - для частих змін структури послідовності.

10. Що таке асоціативний контейнер?

Асоціативний контейнер - це контейнер, у якому елементи зберігаються у вигляді ключів або пар ключ-значення, а доступ до даних здійснюється за ключем. До таких контейнерів належать map, multimap, set та multiset.

11. Навіщо у контейнерах map зберігаються пари?

У контейнері map кожний елемент подається парою: ключ і відповідне йому значення. Саме тому в map використовують pair, де поле first відповідає ключу, а поле second - значенню. Такий підхід забезпечує зручний пошук даних за ключем і підтримує структуру асоціативного контейнера.

12. Що означає суфікс _if?

Суфікс _if у назвах алгоритмів стандартної бібліотеки означає, що алгоритм працює не з конкретним значенням, а з умовою, заданою предикатом. Наприклад, find_if шукає перший елемент, який задовольняє умову, а count_if підраховує кількість елементів, для яких ця умова істинна.

13. Що таке ітератор? Наведіть приклади.

Ітератор - це об'єкт, який забезпечує доступ до елементів контейнера і поводиться подібно до вказівника. Для нього визначені операції розіменування, переходу до наступного елемента і порівняння. Приклади: vector::iterator, list::iterator, map::iterator.

14. Що таке функціональний об'єкт?

Функціональний об'єкт, або функтор, - це об'єкт класу, для якого перевантажено operator(). Завдяки цьому такий об'єкт можна викликати подібно до звичайної функції і передавати в алгоритми стандартної бібліотеки.

15. Чим відрізняється алгоритм count_if від find_if?

Алгоритм count_if підраховує кількість елементів, що задовольняють заданому предикату. Алгоритм find_if знаходить перший елемент, для якого предикат повертає істину, і повертає ітератор на нього.

16. Що робить алгоритм for_each?

Алгоритм for_each послідовно застосовує вказану користувачем функцію або функціональний об'єкт до кожного елемента контейнера у заданому діапазоні. Його використовують для обробки, виведення, накопичення або зміни елементів колекції.

Практичні приклади у середовищі Visual Studio

Рисунок 1 - Код програми з демонстрацією класів string і vector

Visual Studio

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
    string s1 = "Hello";
    string s2 = "Containers";
    string s3 = s1 + " " + s2;

    cout << "s1 = " << s1 << endl;
    cout << "s2 = " << s2 << endl;
    cout << "Concatenation: " << s3 << endl;
    cout << "Length of s3 = " << s3.length() << endl;

    vector<int> numbers;
    numbers.push_back(5);
    numbers.push_back(10);
    numbers.push_back(15);

    cout << "Vector elements: ";
    for (int i = 0; i < (int)numbers.size(); i++) {
        cout << numbers[i] << " ";
    }
    cout << endl;

    cout << "Vector size = " << numbers.size() << endl;
```

Рисунок 2 - Результат виконання програми для string і vector

Visual Studio

```
s1 = Hello
s2 = Containers
Concatenation: Hello Containers
Length of s3 = 16
Vector elements: 5 10 15
Vector size = 3
Vector capacity = 3
```

Практичні приклади у середовищі Visual Studio

Рисунок 3 - Код програми з демонстрацією контейнерів list і map

Visual Studio

```
#include <iostream>
#include <list>
#include <map>
#include <string>
using namespace std;

int main() {
    list<int> values;
    values.push_back(10);
    values.push_back(20);
    values.push_back(30);

    cout << "List elements: ";
    for (int x : values) {
        cout << x << " ";
    }
    cout << endl;

    map<string, int> grades;
    grades["Math"] = 90;
    grades["Physics"] = 85;
    grades["Programming"] = 100;

    cout << "Map elements:" << endl;
    for (auto item : grades) {
        cout << item.first << " -> " << item.second << endl;
    }
```

Рисунок 4 - Результат виконання програми для list і map

Visual Studio

```
List elements: 10 20 30
Map elements:
Math -> 90
Physics -> 85
Programming -> 100
```

Практичні приклади у середовищі Visual Studio

Рисунок 5 - Код програми з демонстрацією алгоритмів find_if, count_if та for_each

Visual Studio

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class PrintValue {
public:
    void operator()(int x) const {
        cout << x << " ";
    }
};

bool isEven(int x) {
    return x % 2 == 0;
}

int main() {
    vector<int> numbers = {1, 2, 3, 4, 5, 6};

    cout << "All elements: ";
    for_each(numbers.begin(), numbers.end(), PrintValue());
    cout << endl;

    int evenCount = count_if(numbers.begin(), numbers.end(), isEven);
    cout << "Count of even numbers = " << evenCount << endl;

    auto it = find_if(numbers.begin(), numbers.end(), isEven);
```

Рисунок 6 - Результат виконання програми для алгоритмів STL

Visual Studio

```
All elements: 1 2 3 4 5 6
Count of even numbers = 3
First even element = 2
```